Project Design Phase-II
Data Flow Diagram & User Stories

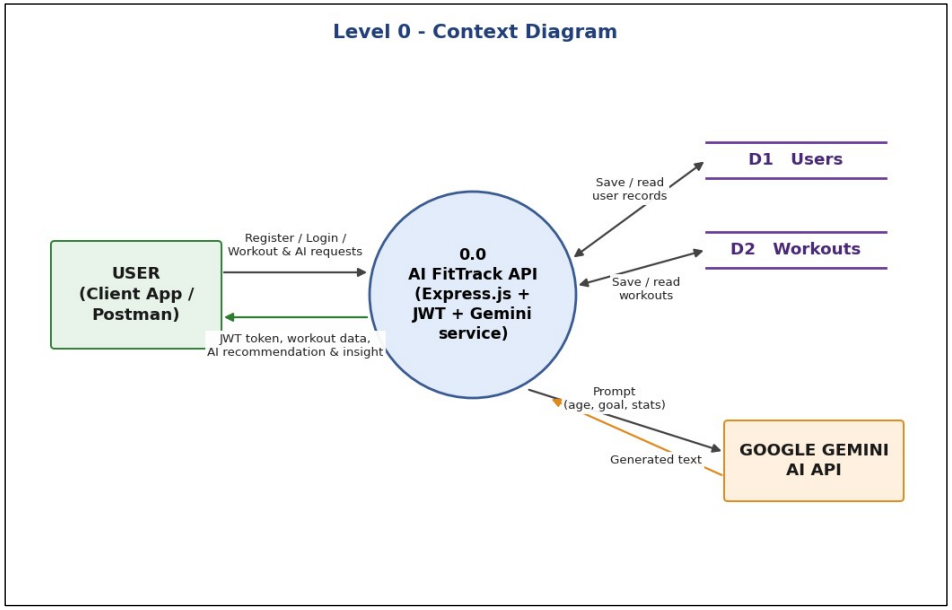
Date	30 September 2026
Team ID	SWTID-2026-2365
Project Name	AI FitTrack API – Fitness Tracking API with Google Gemini AI
Maximum Marks	4 Marks

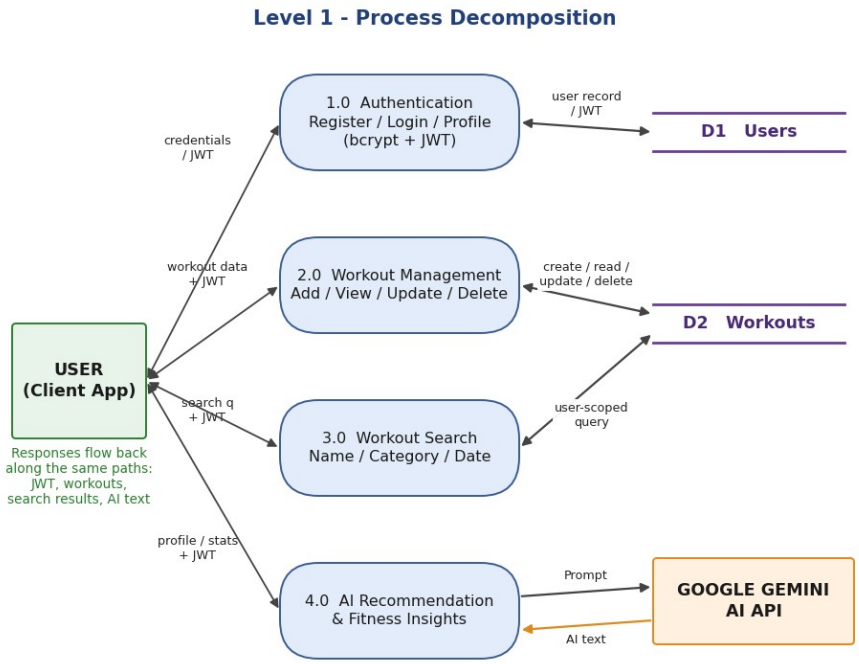
Data Flow Diagrams:

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It shows how data enters and leaves the system, what changes the information, and where data is stored.

Level 0 and Level 1 Data Flow Diagrams – AI FitTrack API:

Level 0 – Context Diagram (AI FitTrack API)





User Stories

Use the below template to list all the user stories for the product.

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
Administrator / Developer	Project Setup	USN-1	As a developer, I can set up the Express server, MongoDB Atlas connection and MVC folder structure so that the API can run.	Server starts on the configured port and logs “MongoDB Connected”	High	Sprint-1
Customer	Registration	USN-2	As a user, I can register for the application by	Account is created and	High	Sprint-1

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
(Mobile / Web user)			entering my name, email and password.	API returns 201 with success = true		
Customer (Mobile / Web user)	Registration	USN-3	As a user, I want my password stored securely and duplicate emails rejected, so that my account is protected.	Password is stored as a bcrypt hash; registering an existing email returns 400	High	Sprint-1
Customer (Mobile / Web user)	Registration	USN-4	As a user, I receive a JWT token as soon as I register, so that I can use the application immediately.	Response contains a signed token valid for 30 days	High	Sprint-1
Customer (Mobile / Web user)	Login	USN-5	As a user, I can log into the application by entering my email and password.	Valid credentials return 200 with a JWT token; invalid credentials return 401	High	Sprint-1
Customer (Mobile / Web user)	User Profile	USN-6	As a user, I can view my profile (id, name, email) using my token.	GET /api/auth/profile returns my details when a valid token is sent	Medium	Sprint-1
Administrator / Developer	Security	USN-7	As a developer, I can protect routes with a JWT middleware so that only authenticated users reach private endpoints.	Missing or invalid token returns 401 “Not authorized”	High	Sprint-1
Customer (Mobile / Web user)	Workout Management	USN-8	As a user, I can add a workout with name, category, duration, calories burned and date.	Workout is saved against my user id and API returns 201	High	Sprint-2
Customer (Mobile / Web user)	Workout Management	USN-9	As a user, I can view all my workouts sorted by latest date.	GET /api/workouts returns count and my workouts in descending date order	High	Sprint-2
Customer (Mobile / Web user)	Workout Management	USN-10	As a user, I can view a single workout by its ID.	Returns the workout if it is mine, otherwise 404	High	Sprint-2

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
Customer (Mobile / Web user)	Workout Management	USN-11	As a user, I can update an existing workout.	Updated workout is returned with validation applied	High	Sprint-2
Customer (Mobile / Web user)	Workout Management	USN-12	As a user, I can delete a workout I no longer need.	Workout is removed and a success message is returned	Medium	Sprint-2
Customer (Mobile / Web user)	Data Privacy	USN-13	As a user, I can only see, edit or delete my own workouts.	Another user's workout ID returns 404 "Workout not found"	High	Sprint-2
Administrator / Developer	Data Validation	USN-14	As a developer, I can validate workout data (supported category, minimum duration, non-negative calories).	Invalid data returns 400 with a clear message	Medium	Sprint-2
Administrator / Developer	Error Handling	USN-15	As a developer, I can handle CastError, duplicate key and validation errors in one central error handler.	All errors return JSON in the form { success: false, error }	Medium	Sprint-2
Customer (Mobile / Web user)	Workout Search	USN-16	As a user, I can search my workouts by name keyword.	GET /api/workouts/search? q=jog returns matching workouts	High	Sprint-3
Customer (Mobile / Web user)	Workout Search	USN-17	As a user, I can search my workouts by category.	?q=running returns workouts of the Running category	Medium	Sprint-3
Customer (Mobile / Web user)	Workout Search	USN-18	As a user, I can search my workouts by date (YYYY-MM-DD).	?q=2026-06-26 returns workouts logged on that day	Medium	Sprint-3
Administrator / Developer	AI Integration	USN-19	As a developer, I can integrate the Google Gemini SDK through a separate service layer	Gemini client initialises from GEMINI_API_KEY	High	Sprint-3

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
			using environment variables.	and model configuration		
Customer (Mobile / Web user)	AI Recommendation	USN-20	As a user, I can enter my age, fitness goal and experience level and receive a personalised workout recommendation.	API returns { recommendation } in 2–3 paragraphs of plain text	High	Sprint-3
Customer (Mobile / Web user)	AI Insights	USN-21	As a user, I can send my workout totals (workouts, average duration, calories) and receive a short AI fitness insight.	API returns { insight } in 2–3 sentences	High	Sprint-3
Administrator / Developer	Testing	USN-22	As a developer, I can test every endpoint through a Postman collection that stores the token and workout ID automatically.	Collection runs Register → Login → Workout CRUD → Search → AI requests	Medium	Sprint-4
Administrator / Developer	Documentation	USN-23	As a developer, I can read a README with setup steps and API specifications.	README lists setup, .env variables and every endpoint with sample request / response	Medium	Sprint-4
Administrator / Developer	System Health	USN-24	As a developer, I can call a health-check endpoint and get a JSON 404 for unknown routes.	GET / returns “API is running successfully”	Low	Sprint-4
Administrator / Developer	Testing	USN-25	As a team, we can carry out user acceptance testing and log bugs for every module.	UAT test cases executed and bugs recorded with severity and status	High	Sprint-4
Administrator / Developer	Security	USN-26	As a developer, I can keep secrets in a .env file that is git-ignored and enable CORS and request logging.	Secrets are not committed; requests are logged by Morgan	High	Sprint-4
Administrator / Developer	Documentation	USN-27	As a team, we prepare the phase-wise project documents for final submission.	All phase documents are completed and submitted	Medium	Sprint-4